

SpanLease

Pre-timeout localization of persistent RPC/executor circular waits from OpenTelemetry-compatible runtime events.

A bounded-scope systems proposal: event contract, causally consistent capacity-aware reconstruction, conservative verdict semantics, and a comparative evaluation plan.

CENTRAL CLAIM

SpanLease is an OpenTelemetry-compatible runtime monitor for synchronous Java/gRPC services using explicitly configured bounded executors. It emits invocation, queue-arrival, executor-slot ownership, blocking-wait, release, cancellation and renewable-liveness events; reconstructs a causally consistent, capacity-aware wait-for graph; and localizes persistent circular waits before RPC deadlines. When observations do not justify confirmation, it returns an inconclusive result rather than imputing missing dependencies.

PREPARED BY	Shashank
PROGRAMME	B.Tech CSE (Data Science), VIT — SCOPE
COURSE / CODE	
FACULTY GUIDE	
TARGET VENUES	ACM Middleware · ICPE · IEEE IC2E
STATUS	Design proposal. No implementation or measurement yet; unproven results are labelled
Labels used throughout: TO BE IMPLEMENTED TO BE PROVED TO BE MEASURED VERIFY CITATION — no result is asserted without one of these unless it follows from published work or a public specification.	
DATE	August 2026

2 · ABSTRACT

Abstract

Microservice systems built on synchronous remote procedure calls can enter persistent circular waits that involve both in-flight RPC executions and bounded executor capacity. Two classes of evidence are available while such a condition persists, and neither is sufficient. Completed-span traces carry full request identity but ordinarily arrive only after the stalled invocations reach their deadlines, and aggregate saturation metrics arrive continuously but cannot identify which executions and which capacity units constitute the responsible cycle.

This proposal describes SpanLease, an OpenTelemetry-compatible runtime monitor for a deliberately narrow setting: synchronous blocking unary gRPC over Java blocking stubs, servers using explicitly configured bounded executors, and AND-wait request semantics. SpanLease reuses standard OpenTelemetry context propagation, the logs data model and OTLP transport, but defines additional event schemas of its own; it is therefore built on OpenTelemetry-compatible *custom* telemetry rather than on standard telemetry alone.

The monitor emits ten event types covering invocation dispatch and arrival, queue entry, executor-slot acquisition and release, blocking-wait boundaries, renewable liveness assertions, cancellation and execution termination. A central analyzer reconstructs a candidate global state that is causally consistent with respect to observed send/arrival relations and local event order, models each executor slot as a distinct resource instance, and evaluates a capacity-aware deadlock predicate over that state. Verdicts are drawn from a four-valued lattice that distinguishes an established predicate, a refuted predicate, a suggested-but-unsupported candidate, and insufficient observability. Detection confidence is kept separate from remediation safety; a confirmed verdict is not by itself a licence to cancel work.

Correctness is stated conditionally. A safety obligation asserts that a confirmed verdict implies a qualifying capacity-aware deadlock at some causally consistent cut within the reported persistence interval, under explicitly stated assumptions. A conditional liveness obligation asserts eventual detection when a qualifying deadlock outlives the detection threshold and the required observations arrive within an assumed delay bound. Both are proof obligations at this stage **TO BE PROVED**.

The evaluation plan compares SpanLease against a metrics-only saturation detector, post-timeout completed-trace diagnosis, an in-progress-span pipeline feeding the same analyzer, an RPC-event wait-for monitor in the Cheriton-Skeen tradition, a DDMon-inspired observer, and triggered-tracing buffering, together with capacity-unaware, lease-free and causality-free ablations. Results are **TO BE MEASURED**.

3 · PROBLEM AND MOTIVATION

Problem and motivation

3.1 Three conditions that must not be conflated

Operational discussion of microservice stalls routinely merges three distinct conditions. The proposal targets only the third, and the distinction governs every design decision that follows.

CONDITION	DEFINITION	RESOLUTION BEHAVIOUR
Slow request	An execution that is making progress, or is waiting on a dependency that will respond, but whose latency is elevated relative to expectation.	Resolves without intervention. Cancelling it destroys useful work.
Executor saturation or starvation	All capacity units of a bounded executor are occupied and arriving invocations queue. A statement about resource pressure.	May resolve as soon as any owner completes. Saturation is <i>evidence of pressure, not proof of deadlock</i> .
Persistent circular wait	A set of executions and capacity units such that every execution in the set is blocked, every capacity unit that could satisfy those blocked executions is held within the set, and no member can progress without an external event.	Does not resolve without intervention. This is the target condition.

Saturation and circular waiting are frequently coincident but are not equivalent in either direction. A saturated executor whose owners are all progressing is healthy under load. A circular wait can also form among executions holding capacity in executors that are not fully saturated, if the specific units required by the blocked members are held within the cycle. A detector that treats saturation as its predicate will therefore produce both false positives and, in less common configurations, false negatives.

3.2 A motivating execution

Three services, each with a configured bounded executor. Slots are named explicitly, because service-level naming is insufficient to state the condition: the predicate is about which *capacity units* are held by which *executions*.

MOTIVATING TIMELINE — EXECUTIONS, SLOTS AND THE INTERVENTION WINDOW

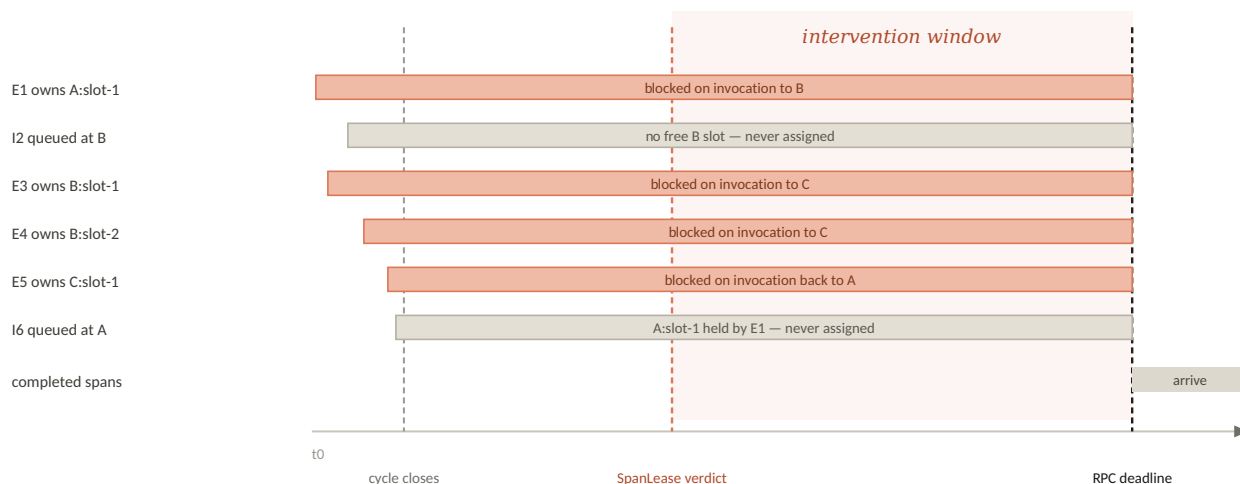


Figure 1. Executions *E1*, *E3*, *E4* and *E5* each hold a named capacity unit and block on a downstream invocation; invocations *I2* and *I6* are queued behind units held inside the set. Every capacity unit capable of serving the queued invocations is held within the set, so the condition is permanent. Completed spans reach the backend only at the deadline. The shaded region is the interval in which a verdict has operational value.

Stated in terms the analyzer will use: *E1* owns *A:slot-1* and waits for invocation *I2*; *I2* is queued at *B* and requires any unit of *B*'s executor; the units *B:slot-1* and *B:slot-2* are owned by *E3* and *E4*, which wait on invocations served only by *C*; *C:slot-1* is owned by *E5*, which waits on invocation *I6* queued at *A*, requiring a unit owned inside the set. The closure is over slots, not services.

3.3 Why the two available evidence classes are each insufficient

EVIDENCE CLASS	TIMING	LIMITATION FOR THIS PREDICATE
Completed spans on a conventional exporter	Ordinarily on span end, so decisive spans arrive at the deadline.	Full request identity, but outside the intervention window in a default configuration.
Aggregate executor and in-flight metrics	Continuous, on a periodic export interval.	Reports counts and occupancy. Cannot name which execution holds which unit, nor which invocation waits on which execution, so it cannot evaluate a closure predicate over specific units.

A CLAIM THIS PROPOSAL DOES NOT MAKE

It is **not** true that distributed tracing in general requires completed executions. AWS X-Ray accepts in-progress segments; Pydantic Logfire emits pending spans; OpenTelemetry has an in-process `zpages` facility that lists currently running spans, and live- and incomplete-span export has been discussed in the specification for several years; Hindsight (NSDI 2023) retains data locally and materialises it retrospectively on a trigger. The accurate statement is narrower: *the default simple and batch span processors export on span end*, so a conventional deployment does not surface the decisive open invocations pre-timeout. Section 12 compares against in-progress-span mechanisms directly rather than dismissing them.

3.4 Cost of detecting only at the deadline

While the condition persists, every capacity unit inside the cycle is unavailable to unrelated traffic, so queueing spreads outward; timed-out clients typically retry, adding load to a system with reduced effective capacity; and callers of the affected services exhaust their own executors in turn. Systems in this regime can remain degraded after the original trigger is removed, requiring external intervention. The operational value of a verdict is therefore not that it arrives sooner but that it identifies *which* executions and units to act on — an action that neither a shortened deadline nor a service restart can express.

4 • SCOPE AND ASSUMPTIONS

Scope, assumptions and non-goals

4.1 Setting

- Synchronous blocking unary gRPC. The wait is explicit rather than inferred.
- Java client and server, using generated blocking stubs.
- Servers configured with an **explicitly bounded executor** whose capacity and slot identities are known to the instrumentation.
- AND-wait request semantics: a blocked execution requires exactly the one outstanding response, and no alternative event unblocks it.
- Trace and invocation context propagated correctly on every hop.

EXTERNAL VALIDITY, STATED HONESTLY

A default gRPC Java deployment does not necessarily match this configuration. Server executors may be unbounded or cached, and capacity may be elastic. The proposal's claims apply to deployments configured with bounded executors as described, and the evaluation must report this as a restriction on generalisability rather than treating the testbed as representative of gRPC deployments in general.

4.2 Assumptions, labelled

ID	ASSUMPTION	CONSEQUENCE IF VIOLATED
A1	Instrumented resources only. Every capacity unit participating in a confirmable cycle emits acquire/release events.	A cycle through an uninstrumented lock, connection pool or broker cannot be confirmed; the analyzer must degrade to an inconclusive verdict rather than assume absence.
A2	Context propagation is correct and invocation identity is stable end to end.	Wait edges cannot be attributed; verdicts are unsound. Detectable in part by coverage accounting (Section 9.3).
A3	Observations on the priority channel may be delayed, reordered or duplicated, but delivery delay for events used in a confirmation is bounded by Δ .	Confirmation is withdrawn; only inconclusive verdicts are permitted.
A4	Physical clock skew between instrumented hosts is bounded by ε . Physical time is used only for persistence thresholds, never for cross-host ordering.	Persistence intervals widen by 2ε ; ordering conclusions are unaffected because they rest on causal relations.
A5	Executor capacity and slot identity are observable and stable for the duration of an evaluation window.	The capacity-aware predicate cannot be evaluated; falls back to inconclusive.
A6	Instrumentation is correct and does not itself drop terminal events under load.	Held-forever ownership edges would be inferred from missing releases. Mitigated by lease expiry, not eliminated by it.

4.3 Non-goals

- No claim for reactive, asynchronous, streaming, actor-based or virtual-thread systems.
- No claim of soundness under arbitrary event loss, unbounded delay or unsynchronized clocks.
- No detection of livelock, starvation without circular waiting, or application-level logical dependencies that are not represented by an observed wait.
- No remediation policy. The system produces evidence; acting on it is a separate problem with separate correctness requirements (Section 9.4).
- No claim to operate on standard OpenTelemetry telemetry alone. The system defines custom event schemas (Section 6).

System and observation model

5.1 Preliminaries

This subsection fixes classical terminology, because the predicate in Section 8 is a capacity-aware refinement of it rather than a new invention.

Deadlock and the Coffman conditions

A deadlock is a state in which every member of a set of processes is waiting for an event that only another member can produce. Coffman, Elphick and Shoshani established four necessary conditions: mutual exclusion over at least one resource; hold-and-wait, in which a process holding a resource requests another; no preemption, so resources are released only voluntarily; and circular wait, a closed chain of dependencies. Each maps onto the present setting: an executor slot serves one execution at a time; a blocked execution retains its slot for the whole wait; the runtime does not reclaim a slot from an in-flight execution spontaneously; and the chain closes when the units required by blocked members are held inside the set.

The third mapping is the one practitioners find least intuitive. A synchronous request handler occupying a bounded executor slot is holding a non-shareable, non-preemptible resource while waiting, which is precisely the Coffman sense. Executor exhaustion is therefore a resource deadlock, not merely a performance phenomenon.

Resource-allocation and wait-for graphs

A resource-allocation graph has process and resource nodes, request edges from process to resource, and assignment edges from resource to process. A wait-for graph contracts resources away, leaving process-to-process edges. The contraction is convenient but lossy: it discards which unit is contended, and that is exactly the information a capacity-aware predicate needs. SpanLease therefore retains resource-instance nodes and does not operate on a contracted wait-for graph.

Why cycle detection alone is wrong here

For single-instance resources, a cycle in the wait-for graph is necessary and sufficient. For resources with capacity greater than one, a cycle is **necessary but not sufficient**: a third holder outside the cycle may be about to release a satisfying unit. An executor with capacity k is a multi-instance resource with k units. Applying ordinary directed-cycle detection to services or to a contracted graph therefore produces false positives whenever spare capacity exists, which is the common case. Section 8 addresses this by modelling each slot individually.

AND versus OR waits

Under AND semantics a blocked party needs all outstanding requests satisfied, and a cycle is the relevant structure. Under OR semantics any one grant suffices and deadlock requires a knot rather than a cycle. A blocking unary RPC is an AND wait with a single element, which is

why the scope is restricted to it; hedged, streaming and quorum patterns are OR or AND-OR and are out of scope.

Phantom cycles and consistent cuts

A monitor assembling edges observed at different times can construct a cycle whose edges never coexisted. Guarding against this requires reasoning about a **consistent cut**: a collection of local states closed under the causal precedence relation, such that no receive is included without its send. Any assertion about the global system must be an assertion about some consistent cut, not about an intuitive "now."

5.2 Entities

Service instance. A single addressable process, identified by `instance_id`, with its own local event sequence.

RPC invocation. A logical downstream call, identified by a stable `invocation_id` generated at the client and propagated in request metadata. The invocation is the unit that links a client-side wait to a server-side arrival, and it exists independently of any span. Both endpoints report the same `invocation_id`.

Execution. The server-side processing entity that can own capacity units and can block, identified by `execution_id`. An execution comes into being when an invocation is assigned a capacity unit. It is *not* the same thing as a span: a span is a representation of an observation, whereas an execution is a resource-holding entity in the model. The two are correlated but must not be used interchangeably.

Queue entry. An invocation that has arrived at a service instance but has not yet been assigned a capacity unit. It has an arrival event and a resource-wait event, but no `execution_id` and no application span — a queued invocation is invisible to span-based instrumentation, which is a specific reason the event contract begins before span creation.

Resource type and resource instance. A resource type is a bounded executor or pool, identified by `resource_id`, with declared capacity k . A resource instance is one capacity unit, identified by `resource_instance_id` (for example `svcB:exec-main:slot-2`). Modelling units rather than types is what makes the predicate in Section 8 correct for $k > 1$.

Wait edge. An observed blocking dependency, either execution-to-invocation (a blocking call) or queue-entry-to-resource-type (awaiting any satisfying unit).

Ownership edge. A resource instance held by an execution, asserted by an acquire event with no causally later release.

Lease. Evidence that an execution, or a wait assertion, remained live during a bounded interval. A lease **bounds the staleness** of an assertion. It does not by itself prevent phantom cycles: delayed or reordered leases can still be combined into a state that never existed, and only the causal-consistency construction of Section 7 rules that out.

Observation. An event record that may be delayed, reordered, duplicated or lost. The analyzer's input is a set of observations, never the system state itself.

Candidate global state. A set of per-instance local states forming a consistent cut under the causal relation of Section 7, over which the predicate of Section 8 is evaluated.

5.3 Observation model

Each service instance emits events onto a dedicated priority channel. The channel is deliberately exempt from the head-based sampling applied to ordinary spans, so **ordinary trace sampling is not the detector's input model**. Two consequences follow, and both must be carried into the evaluation. First, loss characteristics on the priority channel are a property of that channel and must be measured separately rather than inherited from span-sampling literature. Second, sampled completed spans are supplementary explanation for an operator and are never an input to a verdict.

Events may be delayed up to Δ for confirmation purposes (A3), duplicated, or reordered in transit. Duplicates are idempotent by construction, since every event carries `(instance_id, local_seq)`. Loss is not assumed away: an event that never arrives leaves a gap that the coverage accounting of Section 9.3 must expose.

6 · EVENT CONTRACT

The SpanLease event contract

6.1 What is reused and what is new

REUSED FROM OPENTELEMETRY

NEW IN SPANLEASE

W3C trace context propagation; the logs data model with trace and span correlation fields; OTLP transport; collector pipelines and resource attributes; the `SpanProcessor` extension point.

Ten event schemas with their own semantics and required fields; a stable `invocation_id` propagated as request metadata; `execution_id` and `resource_instance_id` identity; per-instance monotonic `local_seq`; a causal-parent reference; and a schema version field.

TERMINOLOGY CORRECTION CARRIED THROUGH THE DOCUMENT

The system is described throughout as using **OpenTelemetry-compatible custom telemetry**, never as using standard telemetry. It rides standard APIs and transport but its event vocabulary is its own. Relatedly, the baseline in Section 11 must not attribute an `rpc.server.active_requests` metric to the OpenTelemetry RPC semantic conventions: current conventions do not standardise such a metric, so any active-request or executor-depth signal used as a baseline is described as custom, framework-specific, or derived from JMX VERIFY CITATION.

6.2 Event types

EVENT	EMITTED BY / WHEN	DISTINGUISHING FIELDS
rpc.invocation.sent	Client interceptor, at dispatch of a blocking unary call.	invocation_id, caller execution_id, target service
rpc.invocation.arrived	Server transport interceptor, on receipt — <i>before</i> any executor assignment or application span.	invocation_id, arrival local_seq
resource.wait.begin	Server, when an arrived invocation is enqueued because no unit is free.	invocation_id, resource_id, requested capacity semantics
resource.acquire	Server, when a unit is assigned. This event creates the execution.	execution_id, resource_instance_id, invocation_id
rpc.block.begin	Client interceptor within a server execution, when the calling thread parks.	execution_id, invocation_id (the awaited call)
execution.lease	Background scanner, after an age threshold and then at a renewal interval, while an execution remains blocked.	execution_id, original start reference, observed age, expiry
rpc.block.end	Client interceptor, on response, error or deadline.	execution_id, invocation_id, outcome
resource.release	Server, when a unit returns to the pool.	execution_id, resource_instance_id
execution.end	Server, on normal termination of the execution.	execution_id, outcome
execution.cancel	Server or client, on cancellation or deadline propagation.	execution_id, invocation_id, cancellation source

DESIGN CORRECTION: AN AGE-THRESHOLD EVENT IS NOT A START EVENT

Emitting nothing until a call exceeds an age threshold keeps volume low, but the record produced at that point is *not* the original start event and must not be labelled as one. The contract therefore separates two things. Lightweight arrival and acquisition events (`rpc.invocation.arrived` , `resource.acquire`) are emitted immediately and are cheap. The delayed record is `execution.lease` , a live assertion that explicitly carries the original start reference and the observed age, so the analyzer never has to infer a start time from the arrival time of a delayed record.

6.3 Common schema

FIELD	TYPE AND MEANING
schema_version	Integer. Permits contract evolution without ambiguous parsing.
event_type	One of the ten types in Section 6.2.
instance_id	Emitting service instance.
local_seq	Monotonic per-instance counter. Establishes local order only — explicitly not a cross-host ordering device (Section 7.1).
causal_parent	Reference to the observation that causally precedes this one across a process boundary, typically an <code>invocation_id</code> plus the peer's emitting sequence where available.
execution_id	Present on every event scoped to a server-side execution.
invocation_id	Stable across client and server for one logical call.
resource_id	Executor or pool identity, with declared capacity.
resource_instance_id	Individual capacity unit. Required on acquire and release.
trace_id, span_id	Standard correlation, so an operator can pivot from a verdict into whatever trace data exists.
wall_time	Local physical timestamp, used only for persistence thresholds under bounded skew ϵ .
lease_expiry	On lease events. Bounds the staleness of the assertion.

TO BE IMPLEMENTED Interceptor placement is a nontrivial engineering question. `rpc.invocation.arrived` must be emitted at the transport layer, before executor dispatch, because a queued invocation has no execution and no application span; standard span-based interceptors run too late to observe it. The proposal's implementation plan (Section 10) treats this as the first integration risk.

Causal and temporal reconstruction

7.1 What sequence numbers do and do not provide

CORRECTION TO AN EARLIER FORMULATION

An earlier draft asserted that per-instance monotonic sequence numbers make cross-host temporal reasoning meaningful without clock synchronisation. That is wrong. Sequence numbers establish *local* order within one instance and nothing more. Ordering two events on different hosts requires either a causal relation between them or a bounded-skew assumption about physical time.

7.2 The chosen model

The proposal adopts a **consistent-cut construction from RPC send/arrival relations and local event order**, with physical time used only for persistence thresholds. Hybrid logical clocks were considered and rejected for the first version, on the grounds that the causal relations needed are already carried explicitly by `invocation_id`, so introducing a second ordering mechanism adds implementation surface without adding safety.

Define the precedence relation $\rightarrow$ as the transitive closure of two generators:

1. *Local order*. For events e, f on the same instance, $e \rightarrow f$ if `local_seq(e) < local_seq(f)`.
2. *Message order*. For an invocation I , `rpc.invocation.sent(I)` $\rightarrow$ `rpc.invocation.arrived(I)`, and symmetrically a response event on the server precedes the corresponding `rpc.block.end` on the client.

A set of local states forms a **candidate global state** if it is a consistent cut with respect to $\rightarrow$: for every invocation whose arrival is included, its dispatch is also included. Edges are admitted into an evaluation only if they are simultaneously valid at some such cut. This is the mechanism that excludes phantom cycles; the lease bounds how stale an assertion may be, but the cut construction is what establishes that the assertions could have held together.

7.3 Persistence, using physical time correctly

The predicate additionally requires that the condition persist for at least a threshold τ , which is a statement about physical duration and therefore requires the bounded-skew assumption A4. Persistence intervals are computed with uncertainty expansion: an interval observed as $[a, b]$ on a remote host is treated as $[a-\varepsilon, b+\varepsilon]$ when compared against intervals from other hosts, and the intersection test uses the conservative narrowed overlap. Ordering conclusions never depend on `wall_time`.

7.4 Correctness obligations

SAFETY OBLIGATION (S1) TO BE PROVED

If SpanLease reports `CONFIRMED_DEADLOCK` over a reported evidence core and persistence interval, then under assumptions A1–A6 there exists a candidate global state — a causally consistent cut with respect to $\rightarrow$ — occurring within that persistence interval at which the capacity-aware deadlock predicate of Section 8 holds over the reported core.

CONDITIONAL LIVENESS OBLIGATION (L1) TO BE PROVED

If a qualifying capacity-aware deadlock exists and persists for longer than $\tau + \Delta + 2\varepsilon$, and every observation required to establish the predicate is delivered within delay bound Δ , then SpanLease eventually reports `CONFIRMED_DEADLOCK` over a core contained in the true deadlocked set.

Both statements are **obligations, not established results**. No soundness claim should appear in a manuscript before the proofs exist, and the correct interim framing is that the algorithm is designed to satisfy S1 and L1 under the stated assumptions. A discharged proof of S1 is the single most valuable theoretical deliverable available to this project.

8 · CAPACITY-AWARE PREDICATE

The capacity-aware deadlock predicate and evidence core

8.1 Graph model

The analyzer maintains a directed graph $G(c)$ over a candidate global state c , with three node classes and four edge classes. Resource instances are first-class nodes; there is no contraction to a service-level or execution-only graph.

ELEMENT	MEANING	ESTABLISHED BY
Node: execution e	A server-side resource-holding entity.	<code>resource.acquire</code> with no causally later <code>resource.release</code> or <code>execution.end</code> .
Node: queue entry q	An arrived invocation awaiting any satisfying unit.	<code>rpc.invocation.arrived</code> plus <code>resource.wait.begin</code> .
Node: resource instance u	One capacity unit of resource type R .	Declared capacity plus observed unit identities.
Edge $e \rightarrow q$ or e'	Execution e is blocked on an invocation whose service-side counterpart is q (queued) or e' (running).	<code>rpc.block.begin</code> with no <code>rpc.block.end</code> , joined by <code>invocation_id</code> .
Edge $q \rightarrow R$	Queue entry requires <i>any one</i> unit of resource type R .	<code>resource.wait.begin</code> .
Edge $u \rightarrow e$	Unit u is owned by execution e .	Unreleased <code>resource.acquire</code> plus a live lease.
Membership $u \in R$	Unit belongs to resource type.	Declared configuration.

8.2 The predicate

Let S be a set of executions and queue entries, and let $U(S)$ be the set of resource instances owned by executions in S . S is a **confirmed terminal set** at candidate global state c if all of the following hold.

COND.	REQUIREMENT
C1	Every execution in S is blocked at c : it has an open <code>rpc.block.begin</code> under AND semantics, with no causally later block-end, release, cancel or end event.
C2	Every queue entry in S has an open <code>resource.wait.begin</code> on some resource type R , with no causally later acquire or cancel.
C3	Capacity closure. For each queue entry $q \in S$ waiting on resource type R : every unit $u \in R$ is owned at c , and every owner of every such unit is itself in S . If any unit of R is free, or is owned by an execution outside S , the set is not terminal.
C4	For each blocked execution $e \in S$, the counterpart of its awaited invocation is also in S .
C5	All ownership is non-preemptive, and no progress event for any member appears at or before c in causal order.
C6	Conditions C1–C5 hold continuously over a persistence interval of at least τ , computed under the uncertainty expansion of Section 7.3.

Condition C3 is the substance of the correction. It replaces cycle detection with a closure test over individual capacity units, and it is what makes the predicate correct for $k > 1$: a directed cycle among services can exist while units remain free, and such a configuration is explicitly not terminal. This is the standard multi-instance resource-reduction condition specialised to bounded executors, not a novel formalism.

8.3 Algorithm

```

CONFIRM(observations 0, threshold tau, delay bound Delta):
  c <- LATEST_STABLE_CUT(0, Delta)      # consistent cut, all deps delivered
  G <- BUILD_GRAPH(0, c)                # exec, queue, unit nodes; live edges only
  B <- { blocked executions and waiting queue entries at c }

  # iterative reduction: repeatedly remove members that could still progress
  S <- B
  repeat
    removed <- false
    for x in S:
      if CAN_PROGRESS(x, S, G):          # C3/C4 violated for x
        S <- S \ {x}; removed <- true
    until not removed

  if S is empty: return CONFIRMED_NO_DEADLOCK
  if COVERAGE(S, 0) < full: return CANDIDATE_INCONCLUSIVE # A1/A2 gap
  if PERSISTED(S, tau): return CONFIRMED_DEADLOCK, CORE(S, G)
  else: return CANDIDATE_INCONCLUSIVE

CAN_PROGRESS(x, S, G):
  if x is a queue entry waiting on resource type R:
    return exists u in R such that u is free at c
    or owner(u) not in S
  if x is a blocked execution:
    return counterpart(awaited_invocation(x)) not in S

```

The reduction is the classical multi-instance approach: begin with every blocked party and repeatedly discharge any party that could still be satisfied from outside the set. What remains is terminal. `TO BE IMPLEMENTED`

COMPLEXITY

Let n be blocked executions and queue entries, m the edges among them, and $k_{\max}$ the largest executor capacity. Graph construction is linear in the observations retained for the window. The reduction removes at least one member per outer pass, so it runs in $O(n)$ passes, each pass costing $O(m + n \cdot k_{\max})$ for the capacity checks, giving a worst case of $O(n \cdot (m + n \cdot k_{\max}))$. Worklist-based implementation reduces the practical cost by re-examining only members whose neighbourhood changed. Space is $O(n + m + \text{total capacity units})$. **TO BE MEASURED** for scaling behaviour under realistic instance counts.

8.4 Partially observed ownership

If some unit of a resource type in the candidate set has no observed owner and no observed free state — the case created by assumption A1 being violated, or by a lost acquire event — then C3 cannot be evaluated. The analyzer must not treat an unobserved unit as free (which would suppress a real deadlock) nor as held within the set (which would fabricate one). It returns **CANDIDATE_INCONCLUSIVE** and reports which units were unaccounted for, so the gap is visible rather than silently resolved.

8.5 Evidence core

The reported core is the terminal set S surviving reduction, together with the ownership and wait edges that witness C3 and C4. This is an **irreducible** core: no member can be removed without violating a condition, because the reduction has already discharged every removable member. It is *not* claimed to be a globally minimum-cardinality core; computing a minimum witness is a separate optimisation problem and no such claim is made **TO BE PROVED** if a minimality result is later desired. The evaluation in Section 11 therefore measures accuracy against the ground-truth deadlocked set using node and edge precision and recall and Jaccard overlap, rather than asserting minimality.

9 · VERDICT SEMANTICS

Verdict semantics

9.1 A four-valued result lattice

The informal three-label scheme of the earlier draft is replaced with four values that are distinguished by *what the evidence establishes*, not by degree of confidence.

VERDICT	FORMAL CONDITION
CONFIRMED_DEADLOCK	A confirmed terminal set exists at a candidate global state, all of C1–C6 hold, and observation coverage over the participating resource types is complete under A1–A6.
CONFIRMED_NO_DEADLOCK	For the queried cut and window, the reduction empties the candidate set: every blocked party is discharged by a free or externally owned unit, or by a counterpart outside the set. The predicate is <i>refuted</i> , not merely unestablished.
CANDIDATE_INCONCLUSIVE	A non-empty blocked set survives reduction, but at least one required observation is missing, delayed beyond Δ , or attributable to an unobserved unit; or persistence has not yet reached τ . A cycle is suggested; the predicate is not established.
INSUFFICIENT_OBSERVABILITY	Coverage is too incomplete to evaluate the predicate at all — for example a participating resource type is uninstrumented, or context propagation is absent on a hop, so no meaningful candidate set can be constructed.

EVIDENCE LATTICE — UPWARD MEANS MORE ESTABLISHED



Figure 2. The two confirmed verdicts are incomparable with each other and both dominate the inconclusive states. Additional coverage or delivered evidence moves a result upward; nothing moves a result from one confirmed verdict to the other without a change in the underlying system state.

9.2 Why the inconclusive states are distinct

CANDIDATE_INCONCLUSIVE means the analyzer *could* evaluate the predicate and found a surviving set whose confirmation is blocked by a specific missing item. It is actionable: the report names the missing observation, so an operator or an adaptive instrumentation policy can obtain it.

INSUFFICIENT_OBSERVABILITY means the predicate is not evaluable in the current deployment at all, which is a configuration defect rather than a transient evidence gap. Collapsing the two into a single "unknown" discards the distinction between "wait for one more lease" and "instrument that connection pool."

9.3 Coverage accounting

Every verdict is accompanied by a coverage report: which resource types in the candidate neighbourhood were instrumented, which units had observed ownership state, which hops carried propagated invocation identity, and which required observations were outstanding at the evaluated cut. Coverage is a first-class output rather than diagnostic logging, because the difference between the verdicts in Section 9.1 is entirely a statement about coverage.

9.4 Detection confidence is not remediation safety

CLAIM REMOVED FROM THE EARLIER DRAFT

The earlier draft stated that a confirmed verdict was safe to act on automatically. That conflates two different properties and is withdrawn. A confirmed terminal set establishes that the predicate held at some causally consistent cut within the persistence interval. It does *not* establish that the condition still holds at the moment a remediation would execute, nor that cancelling any particular member is the correct or least harmful intervention, nor that cancellation will not itself produce inconsistent application state. A remediation policy — victim selection, re-execution semantics, idempotence requirements, and its own safety argument — is out of scope for this proposal and would require separate specification and validation.

10 · IMPLEMENTATION PLAN

Implementation plan

Nothing in this section is built yet. Each item is labelled, and the ordering is chosen so that the two highest-risk integration questions are resolved before any analyzer work begins.

PHASE	DELIVERABLE	EXIT CRITERION
P0	Bounded-executor gRPC testbed: three services, configured fixed-capacity executors with externally visible slot identity, deterministic deadlock trigger, and an independent ground-truth recorder producing request- and slot-level truth. TO BE IMPLEMENTED	The testbed deadlocks on demand and the ground-truth recorder identifies the exact executions and units involved, independently of any SpanLease event.
P1	Transport-layer arrival interception, emitting <code>rpc.invocation.arrived</code> and <code>resource.wait.begin</code> before executor dispatch. TO BE IMPLEMENTED	A queued invocation is observable while it has no execution and no application span. <i>This is the primary integration risk.</i>
P2	Slot-level acquire and release instrumentation with stable <code>resource_instance_id</code> . TO BE IMPLEMENTED	Every unit's ownership state is reconstructible for the whole run; reconstructed occupancy matches the executor's own accounting.
P3	Invocation identity propagation and client-side block boundary events.	Client wait edges join to server counterparts on <code>invocation_id</code> with no unmatched edges under nominal conditions.
P4	Lease scanner with age threshold and renewal interval; priority OTLP channel.	Event volume bounded and characterised as a function of threshold and renewal interval. TO BE MEASURED
P5	Analyzer: consistent-cut construction, graph build, reduction, coverage accounting, verdict emission.	Reproduces ground truth on the P0 testbed; returns <code>CONFIRMED_NO_DEADLOCK</code> for cyclic call patterns with spare capacity.
P6	Baseline implementations (Section 11.2) and ablations.	Every baseline runs on the identical workload with identical ground truth.
P7	Reproducibility package: code, workload definitions, raw results, analysis scripts, seeds.	Clean reproduction of every reported figure on a fresh environment.

11 · EVALUATION

Evaluation questions, baselines, workloads and metrics

11.1 Research questions

RQ	QUESTION
RQ1	Does SpanLease localize qualifying circular waits before the RPC deadline, and how much of the deadline budget is recovered relative to an independent ground-truth onset?
RQ2	Does it separate the three conditions of Section 3.1 — slow progress, saturation without circularity, and persistent circular waiting?
RQ3	How accurately is the evidence core identified at execution and slot granularity?
RQ4	How do verdict distributions behave under event loss, delay, reordering, duplication, clock skew, crashes and partitions?
RQ5	What are the runtime, telemetry and analyzer costs, and how do they scale?
RQ6	What does each mechanism contribute — capacity awareness, leases, causal consistency — as measured by ablation?

11.2 Baselines

BASELINE	WHAT IT ISOLATES
Metrics-only saturation detection custom or JMX-derived executor depth and in-flight counts	Whether continuous aggregate signals alone suffice. Expected to detect pressure without identifying the responsible set; this is the discriminating comparison for RQ3.
Completed-trace and RPC-timeout diagnosis	The status quo. Establishes the detection-latency gap.
In-progress spans plus the same analyzer X-Ray-style in-progress segments or Logfire-style pending spans	The strongest baseline. Isolates the value of the custom event contract from the value of live span visibility, by holding the analyzer constant. If in-progress spans plus this analyzer perform comparably, the contribution reduces to the resource model.
RPC-event wait-for monitor in the Cheriton-Skeen tradition	Whether invocation-level wait-for monitoring without capacity modelling suffices.
DDMon-inspired observer	Comparison against a formally grounded RPC deadlock monitor. Both the proxy and the passive observing variants described in that line of work are considered.
Triggered buffering, Hindsight-style	Whether retrospective materialisation on a trigger achieves the same localisation more cheaply. WHERE FEASIBLE
Perfect-information oracle	Ground truth only; never a competitor.

ABLATIONS

- **Capacity-unaware:** replace C3 with ordinary cycle detection over a contracted graph. Expected to raise false confirmations on cyclic patterns with spare capacity.
- **Lease-free:** assert edges without expiry. Expected to raise false confirmations from stale ownership after lost terminal events.
- **Causality-free:** assemble edges by wall-clock proximity rather than consistent cuts. Expected to raise phantom confirmations under delay and reordering.

11.3 Workloads and fault matrix

DIMENSION	LEVELS
Cycle length	Two, three and five services.
Executor capacity	Several capacities per service, including $k=1$ and $k>1$, since $k=1$ hides the capacity-awareness question entirely.
Replication	Single and multiple instances per service, so that a satisfying unit may exist on a replica outside the cycle.
Offered load	Below, at, and above the point of executor saturation.
Hard negatives	Slow but progressing dependencies; transient saturation; partial pool exhaustion ; cyclic call patterns with spare capacity; retries; hedged calls; cancellation races; garbage-collection pauses; network jitter.
Failure negatives	Service crashes; network partitions; missing <code>execution.end</code> or <code>resource.release</code> events.
Telemetry stress	Priority-channel loss swept across levels; bounded and unbounded delay; reordering; duplication; clock skew up to and beyond ε .
Coverage stress	Cycles routed through an uninstrumented resource, to exercise <code>INSUFFICIENT_OBSERVABILITY</code> .

11.4 Metrics

CATEGORY	MEASURES
Timeliness	Detection latency from independent ground-truth onset; deadline budget saved; fraction of incidents detected inside the intervention window.
Localisation	Node and edge precision and recall at execution and slot granularity; exact-core accuracy; Jaccard overlap with the ground-truth set.
Correctness under uncertainty	False-confirmation rate, reported with a one-sided upper confidence bound even when zero false confirmations are observed ; inconclusive rate by cause; observability coverage.
Cost	CPU, memory, throughput, p99 latency, telemetry bytes per request, analyzer throughput and scaling with instance and unit counts.

STATISTICAL DISCIPLINE

A fixed repetition count is not an adequacy criterion. The number of repetitions per configuration is chosen by a precision analysis targeting a stated confidence-interval half-width for detection latency, and by a power analysis for the comparisons that carry the argument. All point estimates are reported with confidence intervals. Because a zero observed false-confirmation count does not bound the underlying rate, a one-sided upper bound is reported in every such case rather than a claim of zero. Seeds, workload definitions and analysis scripts are published.

12 · RELATED WORK

Related work and explicit comparison

Organised by comparison family rather than chronology. Each family contains work whose existence constrains what this proposal may claim, and the constraint is stated in each case.

12.1 Classical deadlock detection and consistent predicate detection

Chandy, Misra and Haas established distributed edge-chasing detection for resource and communication deadlock; Knapp's survey unified wait-for models, AND and OR semantics, phantom deadlocks and the major algorithm families. Cooper and Marzullo, and Garg and Waldecker, established that assertions about global predicates over distributed computations are naturally modal — *possibly* versus *definitely* over consistent cuts. Runtime-verification theory contributes three-valued semantics with an inconclusive verdict and the notion of monitorability.

Constraint imposed. Neither cycle and knot detection, nor multi-valued verdicts, nor the consistent-cut requirement may be claimed as novel. The lattice of Section 9 is an application of established modal predicate detection, and the manuscript must say so.

12.2 RPC and distributed-object monitors

Cheriton and Skeen's SOSP 1993 critique of causally and totally ordered communication argues for state-level and application-level approaches, and discusses wait-for monitoring at RPC-instance granularity VERIFY CITATION — this specific attribution must be checked against the paper before it is cited in a manuscript. Kaveh and Emmerich (ESEC/FSE 2001) address deadlock detection in distributed object systems. DDMon (OOPSLA 2025) provides correct black-box monitors for distributed deadlock detection, mechanised, with probe-exchanging proxy monitors; the accompanying thesis additionally describes **passive observing monitors**, though the proxy implementation carries the stronger formal guarantees.

Constraint imposed. Request-level or invocation-level RPC wait-for monitoring is not new. Neither is a formally correct RPC deadlock monitor. Crucially, the existence of passive observing monitors in the DDMon line means this proposal cannot claim that passive observation of RPC deadlock is novel either; the distinction must rest on the capacity model, the OTel-compatible event contract, and the coverage-conscious verdict semantics.

12.3 Middleware thread-pool modelling and prevention

Sethi and Anand (Middleware 2008) address concurrency and thread-pool behaviour in enterprise SOA middleware VERIFY CITATION. IBM patent US8060878B2 concerns bounded web-service thread pools and constitutes non-academic prior art VERIFY CITATION.

Constraint imposed. Distributed thread-pool deadlock is a recognised phenomenon with both academic and industrial prior art. This proposal must not claim to be the first treatment of it. The available distinction is the combination of slot-level identity, a formal capacity-closure predicate, and pre-timeout localisation from a telemetry-compatible event stream.

12.4 Live and in-progress observability

AWS X-Ray accepts in-progress segments; Pydantic Logfire emits pending spans; OpenTelemetry provides in-process `zpages` including a running-span view, and live or incomplete span export has been discussed in the specification. Commercial tracers additionally support partial flushing of long-running traces.

Constraint imposed. Exposing incomplete or in-progress spans is not novel, and no claim may suggest otherwise. The strongest baseline in Section 11.2 is deliberately constructed from this family precisely because it is the closest available alternative.

12.5 Triggered and retrospective tracing

Hindsight (NSDI 2023) retains trace data locally at low cost and materialises it retrospectively when a trigger fires. This is architecturally analogous to the two-channel design proposed here.

Constraint imposed. The idea of a cheap always-on capture path plus a selective materialisation path is prior art. The proposal's channel separation is an application of it.

12.6 Local and infrastructure-level approaches

JVM deadlock discovery via thread introspection; executor-starvation detection in application frameworks; database wait-for-graph deadlock detection, which is routine in distributed database engines; kernel-level wait analysis using tracing; and eBPF mutex deadlock detectors that construct live lock wait-for graphs via uprobes.

Constraint imposed. Live wait-for-graph construction is not novel. The scope distinction is cross-service closure over RPC invocations and executor capacity, which per-host and per-engine mechanisms do not address.

12.7 Comparison table

APPROACH	LIVE EVIDENCE	REQUEST IDENTITY	CROSS-SERVICE	MULTI-INSTANCE CAPACITY	PASSIVE TELEMETRY	CAUSAL CONSIST.	LOSS HANDLED	PRE-TIMEOUT VERDICT	OVERHEAD EVAL.
Classical WFG algorithms	assumed	yes	yes	in theory	n/a	yes	no	yes	analytic
Consistent predicate detection	assumed	yes	yes	varies	n/a	yes	no	yes	analytic
Cheriton-Skeen style RPC monitor	yes	yes	yes	no	partly	partly	no	yes	limited
Kaveh & Emmerich	yes	yes	yes	no	no	partly	no	yes	limited
DDMon (proxy)	yes	yes	yes	no	no	yes	no	yes	yes
DDMon (passive observer)	yes	yes	yes	no	yes	weaker	no	yes	partly
Middleware pool modelling	varies	no	partly	yes	varies	no	no	prevent	yes
In-progress spans (X-Ray, Logfire)	yes	yes	yes	no	yes	no	no	partial	vendor
Triggered tracing (Hindsight)	yes	yes	yes	no	yes	no	partly	trigger	yes
Executor / in-flight metrics	yes	no	no	counts	yes	no	n/a	satur. only	yes
eBPF / JVM local detectors	yes	local	no	locks	no	local	no	yes	yes
Completed traces + timeout	no	yes	yes	no	yes	post hoc	sampl.	no	yes
SpanLease (proposed)	yes	yes	yes	slot level	custom OTel	explicit	explicit	yes	planned

"Passive telemetry" distinguishes one-way event export from monitor-to-monitor probe coordination. SpanLease is one-way but not passive in the sense of requiring no new instrumentation.

13 · CONTRIBUTIONS

Contributions and bounded novelty statement

13.1 Contributions, each independently testable

#	CONTRIBUTION	HOW IT IS TESTED
C1	A precise event contract connecting RPC invocations, server executions, queue entries, bounded executor slots and blocking calls through OpenTelemetry-compatible telemetry, including observation of invocations that are queued and therefore have no span.	Reconstruction fidelity against independent slot-level ground truth; unmatched-edge rate; the P1 exit criterion.
C2	A causally consistent, capacity-aware reconstruction algorithm that behaves conservatively under incomplete observation, with stated safety and conditional liveness obligations.	False-confirmation rate with upper bound; capacity-unaware, lease-free and causality-free ablations; discharge of S1.
C3	A pre-timeout, request- and slot-level localisation interface reporting an evidence core, a coverage report, and an explicit inconclusive state distinguishing missing evidence from missing instrumentation.	Core precision, recall and Jaccard overlap; inconclusive rate by cause; coverage-stress scenarios.
C4	A Java/gRPC implementation and controlled comparison against RPC-monitor, in-progress-span, metrics and completed-trace baselines on identical workloads and ground truth.	The comparison itself, per Section 11.
C5	An empirical characterisation of the observability, overhead and detection-latency trade-off as a function of age threshold and renewal interval.	Cost metrics swept across threshold and interval settings.

13.2 Bounded novelty statement

THE CLAIM, PLACED AFTER THE COMPARISON AND NOT BEFORE IT

Individually, none of the following are claimed as novel: wait-for graphs, resource-instance modelling, live or in-progress span exposure, leases, multi-valued verdicts, passive RPC observation, or distributed thread-pool deadlock as a phenomenon. Having established that comparison in Section 12, and to the best of our knowledge, no prior system combines a slot-level executor capacity-closure predicate, a causally consistent reconstruction from OpenTelemetry-compatible custom events including pre-span queue observation, and coverage-conscious verdict semantics that distinguish missing evidence from missing instrumentation, evaluated against in-progress-span and RPC-monitor baselines on identical ground truth.

The qualifier "to the best of our knowledge" appears exactly once, attaches to the integrated system rather than to any component, and follows rather than precedes the comparison — which is the only placement that makes it a defensible statement rather than a rhetorical hedge.

14 · LIMITATIONS

Limitations and threats to validity

Assumption-dependent correctness

Both correctness obligations are conditional on A1–A6. Outside bounded delay Δ , bounded skew ε , and complete instrumentation of participating resource types, no confirmation is justified and the system is designed to say so rather than degrade silently.

Permanent telemetry loss

A permanently lost acquire or release event leaves a unit's ownership state indeterminate. Lease expiry bounds how long a stale assertion survives but does not recover the lost fact. The consequence is an elevated inconclusive rate, which must be reported rather than tuned away.

Hidden resources

A cycle passing through an uninstrumented mutex, connection pool, broker or external service cannot be confirmed. This is the most likely reason for the system to be unhelpful in a real deployment, and it places a hard bound on practical coverage.

Context propagation and instrumentation defects

Absent or incorrect propagation breaks invocation joining; instrumentation that drops terminal events under load produces apparently permanent ownership. Both are detectable in part through coverage accounting, but neither is fully excluded.

Scope restriction and external validity

Results apply to synchronous blocking unary gRPC over Java blocking stubs with explicitly configured bounded executors. Default gRPC Java deployments may not match this configuration, and reactive, streaming, actor-based and virtual-thread systems are outside the model. Synthetic testbed workloads differ from production traffic in arrival process, dependency depth and failure correlation.

Observer effect

The priority channel is unsampled by design, so it adds load precisely when the system is already degraded — the moment its cost is least affordable. Measuring whether the instrumentation perturbs the condition it observes is a required experiment, not an afterthought **TO BE MEASURED**.

Logical dependencies outside the model

Application-level dependencies not represented by an observed blocking wait — a request that will never succeed for semantic reasons, or coordination through shared external state — are invisible. The system detects observed circular waiting, not all forms of permanent non-progress.

15 · PUBLICATION AND ARTIFACT PLAN

Publication and artifact plan

15.1 Venue positioning

VENUE	FIT	WHAT THE SUBMISSION MUST LEAD WITH
ACM Middleware	Strongest thematic fit, once implementation, formal model and evaluation exist. Executor capacity modelling and middleware instrumentation are core to the venue's scope.	The event contract and the capacity-aware predicate as a middleware mechanism, with the comparison against RPC monitors and in-progress-span systems.
ICPE	Suitable if the evidence centres on overhead and the observability/latency trade-off rather than on formal guarantees.	The cost characterisation of Section 11.4 and the threshold/interval sweep.
IEEE IC2E	Suitable if framed as a cloud-engineering diagnostic mechanism.	Deployment practicality, measured overhead, coverage behaviour in realistic configurations.

SUBMISSION READINESS

This proposal should not be submitted as a full research paper in its current state. A credible submission requires an implemented artifact, the strong baselines of Section 11.2 — particularly the in-progress-span baseline sharing the same analyzer — adversarial negative cases, statistical results with confidence intervals and one-sided bounds, and a reproducibility package. A workshop or short paper presenting the event contract and preliminary reconstruction results is the appropriate first step.

15.2 Artifact plan

- Instrumentation library, analyzer, and testbed services as separate reproducible components.
- Workload definitions and fault-injection schedules as declarative configuration, not scripts embedded in prose.
- Independent ground-truth recorder released alongside the detector, since the credibility of every accuracy number depends on it.
- Raw results, seeds and analysis scripts sufficient to regenerate every figure and table.
- Baseline implementations included, so that comparisons are reproducible rather than reported.

16 • REFERENCES

References

Entries marked **VERIFY CITATION** require confirmation of the specific attributed content against the publisher of record before use in a manuscript. No entry below is cited for a claim beyond what appears in the referenced work.

#	REFERENCE
1	E. G. Coffman, M. Elphick and A. Shoshani, "System Deadlocks," <i>ACM Computing Surveys</i> , 3(2), 1971.
2	K. M. Chandy, J. Misra and L. M. Haas, "Distributed Deadlock Detection," <i>ACM Transactions on Computer Systems</i> , 1(2), 1983.
3	E. Knapp, "Deadlock Detection in Distributed Databases," <i>ACM Computing Surveys</i> , 19(4), 1987.
4	R. Cooper and K. Marzullo, "Consistent Detection of Global Predicates," <i>ACM/ONR Workshop on Parallel and Distributed Debugging</i> , 1991.
5	V. K. Garg and B. Waldecker, "Detection of Weak Unstable Predicates in Distributed Programs," <i>IEEE TPDS</i> , 5(3), 1994.
6	D. R. Cheriton and D. Skeen, "Understanding the Limitations of Causally and Totally Ordered Communication," <i>ACM SOSP</i> , 1993. VERIFY CITATION for the RPC-instance wait-for monitoring attribution.
7	N. Kaveh and W. Emmerich, "Deadlock Detection in Distributed Object Systems," <i>ESEC/FSE</i> , 2001.
8	A. Sethi and V. Anand, "On Concurrency Improvements in Enterprise SOA Middleware," <i>ACM/IFIP/USENIX Middleware</i> , 2008. VERIFY CITATION
9	IBM, US Patent 8060878 B2, concerning bounded thread pools for web services. VERIFY CITATION
10	R. J. Rowicki, A. Francalanza and A. Scalas, "Correct Black-Box Monitors for Distributed Deadlock Detection," <i>PACMPL / OOPSLA</i> , 2025. DOI 10.1145/3763069. Technical report arXiv:2508.14851. Associated thesis for the passive observing monitor variant. VERIFY CITATION for the thesis attribution.
11	P. S. Duggirala, T. T. Johnson, A. Zimmerman and S. Mitra, "Static and Dynamic Analysis of Timed Distributed Traces," <i>IEEE RTSS</i> , 2012. DOI 10.1109/RTSS.2012.69.
12	F. Giraldeau and M. Dagenais, "Wait Analysis of Distributed Systems Using Kernel Tracing," <i>IEEE TPDS</i> , 27(8), 2016. DOI 10.1109/TPDS.2015.2488629.
13	L. Zhang et al., "The Benefit of Hindsight: Tracing Edge-Cases in Distributed Systems," <i>USENIX NSDI</i> , 2023.
14	Z. Zhang et al., "CRISP: Critical Path Analysis of Large-Scale Microservice Architectures," <i>USENIX ATC</i> , 2022.
15	A. Bauer, M. Leucker and C. Schallhart, "Runtime Verification for LTL and TLTL," <i>ACM TOSEM</i> , 20(4), 2011.
16	T. D. Chandra and S. Toueg, "Unreliable Failure Detectors for Reliable Distributed Systems," <i>Journal of the ACM</i> , 43(2), 1996. DOI 10.1145/226643.226647.
17	N. Bronson, A. Aghayev, A. Charapko and T. Zhu, "Metastable Failures in Distributed Systems," <i>HotOS</i> , 2021.
18	OpenTelemetry, <i>Tracing SDK Specification</i> : <code>SpanProcessor</code> <code>OnStart</code> / <code>OnEnd</code> and finished-span exporters.
19	OpenTelemetry, <i>Logs Data Model</i> : trace and span correlation fields on log records.
20	OpenTelemetry, <i>Semantic Conventions for RPC</i> . Cited only for span conventions; no active-request metric is attributed to it.
21	OpenTelemetry, <code>zpages</code> including the running-span (<code>tracez</code>) view; specification discussions on live and incomplete span export. VERIFY CITATION
22	Amazon Web Services, <i>AWS X-Ray Developer Guide</i> : in-progress segments. VERIFY CITATION
23	Pydantic, <i>Logfire documentation</i> : pending spans. VERIFY CITATION
24	Datadog tracer documentation: partial flush configuration for long-running traces.

#	REFERENCE
25	IO Visor Project, <code>deadlock_detector</code> BCC tool; Inspektor Gadget deadlock gadget — eBPF live mutex wait-for graphs via uprobes.
26	Z. Liu, S. Zhu, B. Qin, H. Chen and L. Song, "Automatically Detecting and Fixing Concurrency Bugs in Go Software Systems" (GCatch), <i>ASPLOS</i> , 2021.

APPENDIX · PRE-SUBMISSION GATE

Unresolved tasks before submission

Grouped by type. The document is not submission-ready until every item is discharged. Items are drawn from the revision requirements and from the acceptance gate.

PROOF OBLIGATIONS

- ☐ Discharge safety obligation S1: a confirmed verdict implies the capacity-aware predicate at a causally consistent cut within the persistence interval, under A1–A6.
- ☐ Discharge conditional liveness obligation L1 under the delay and persistence bounds.
- ☐ Prove or explicitly disclaim irreducibility of the reported evidence core; do not claim minimum cardinality without proof.
- ☐ State and justify the uncertainty-expansion argument for persistence intervals under bounded skew.
- ☐ Verify that the reduction algorithm terminates and that its fixed point is independent of examination order.

ENGINEERING TASKS

- ☐ Transport-layer arrival interception that observes a queued invocation before executor dispatch and before any application span (P1 — primary integration risk).
- ☐ Stable slot-level `resource_instance_id` assignment and release accounting reconcilable against the executor's own state.
- ☐ Invocation identity propagation with no unmatched client/server edges under nominal conditions.
- ☐ Independent ground-truth recorder producing request- and slot-level truth without reference to SpanLease events.
- ☐ Priority channel with measured loss characteristics of its own, separate from span sampling.
- ☐ Baseline implementations, including the in-progress-span pipeline feeding the identical analyzer.

EXPERIMENTAL TASKS

- ☐ Precision and power analysis to determine repetition counts; abandon any fixed-repetition rule.
- ☐ One-sided upper confidence bounds for false-confirmation rate, including when zero are observed.
- ☐ Capacity-unaware, lease-free and causality-free ablations.
- ☐ Sweep of $k=1$ and $k>1$ capacities, replica counts, and cycle lengths of two, three and five.
- ☐ Hard negatives: partial pool exhaustion, cyclic patterns with spare capacity, retries, hedging, cancellation races.

- ☐ Telemetry stress: loss, delay, reordering, duplication, clock skew beyond ε .
- ☐ Coverage stress producing `INSUFFICIENT_OBSERVABILITY` through an uninstrumented resource.
- ☐ Observer-effect measurement of the unsampled channel under degraded conditions.

CITATION AND CLAIM AUDIT

- ☐ Verify every entry marked `VERIFY CITATION`, especially the Cheriton–Skeen RPC-instance attribution, the Sethi–Anand and IBM patent content, the DDMon passive-monitor variant, and the X-Ray, Logfire and zpages product behaviours.
- ☐ Confirm that no text attributes an `rpc.server.active_requests` metric to OpenTelemetry semantic conventions.
- ☐ Confirm that no text claims tracing in general requires completed executions.
- ☐ Confirm that "standard telemetry" appears nowhere as a description of the system's input.
- ☐ Confirm the single "to the best of our knowledge" qualifier appears only in Section 13.2, after the comparison.
- ☐ Confirm no statement links a confirmed verdict to remediation safety.

CLOSING POSITION

The defensible contribution is an integration and formalisation, not a priority claim. It is a slot-level capacity-closure predicate over a causally consistent reconstruction, built from an OpenTelemetry-compatible event contract that can observe an invocation before it has an execution or a span, with verdict semantics that distinguish missing evidence from missing instrumentation — evaluated against the closest live-observability and RPC-monitoring alternatives on identical ground truth. Every component has ancestry; the composition, the capacity predicate and the coverage-conscious semantics are what the paper must defend.